



UNIVERSIDAD
NACIONAL
DE COLOMBIA

UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingeniería

Matemáticas Discretas II

Optimización de Estructuras de Datos: D-MPHF en Sistemas Embebidos

Samuel Ricardo Pardo Trujillo

Luis Alejandro Sanchez Moreno

Profesor: Arles Ernesto Rodriguez Portela

Bogotá, Colombia - Julio de 2026

1. Introducción	2
2. Objetivos del Proyecto	3
2.1 Objetivo General	3
2.2 Objetivos Específicos	3
3. Marco Teórico Formal y Justificación Matemática	3
3.1 Formalización de la Inyectividad y Biyectividad en el Espacio Discreto	3
3.2 Módulo 1: Teoría de Grupos y Acciones Algebraicas Estrictas	4
3.3 Módulo 2: Teoría de Números, Aritmética Modular y Criptografía	5
4. Análisis Matemático Avanzado mediante Ecuaciones en Diferencias	6
4.1 Modelado Dinámico del Efecto Dominó en Inserciones Dinámicas	6
4.2 Resolución Analítica por el Método de Coeficientes Indeterminados	6
4.3 Análisis Estocástico de Convergencia Exponencial	7
5. Metodología e Ingeniería de Software	8
6. Desarrollo Detallado de Módulos e Implementación	10
6.1 Módulo de Aritmética Modular y Teoría de Números	11
6.2 Módulo de Estructuras Sucintas	11
6.3 Herramienta de Modelado de Recurrencias	12
7. Representación Sucinta y Optimización de Estructuras Bitwise	13
8. Resultados Empíricos y Gráficas de Rendimiento	13
8.1 Tabla Comparativa de Rendimiento Experimental	14
8.2 Gráficas de Rendimiento	14
9. Interpretación Analítica de las Gráficas Resultantes	16
10. Conclusiones y Trabajo Futuro	16
11. Referencias Bibliográficas	16
12. Anexos	17
Estructura Completa del Repositorio	17

1. Introducción

En las ciencias de la computación modernas, la gestión de RAM debido a la forma de su funcionamiento es cada vez más necesaria, por lo cual se requiere la creación de estructuras que optimicen su uso. Las técnicas tradicionales que se usan en muchos entornos suelen tener un problema de colisiones o con el desaprovechamiento de la memoria debido a los factores de carga. El proyecto con nombre **proyecto-discretas-ii-main** ubicado en GitHub, surge como una respuesta fundamentada en las matemáticas presentes en las estructuras discretas.

El propósito de este documento es proporcionar una guía y explicación sobre la planificación, la explicación matemática, los componentes adaptables del software, las estrategias de testeo y la comparación del rendimiento de una implementación nativa en Python de Funciones de Hash

Perfectas Mínimas Concretas (DMPHF) y estructuras de datos de rango.

Este informe está estructurado para guiar al lector desde los fundamentos teóricos matemáticos hasta los detalles de bajo nivel del código fuente. El alcance del proyecto comprende la modelación matemática de grupos, cálculo y resolución de recurrencias lineales para la estimación de costes algorítmicos, aritmética modular, algoritmos de desplazamiento y empaquetamiento para hashes perfectos, validación mediante pruebas de caja negra, y la generación de gráficas estadísticas de rendimiento.

2. Objetivos del Proyecto

2.1 Objetivo General

Diseñar, codificar, validar y someter a pruebas de esfuerzo un sistema informático basado en estructuras algebraicas discretas, con el fin de instanciar Funciones de Hash Perfectas Mínimas Estáticas capaces de organizar conjuntos de datos cerrados con tiempo de consulta instantáneo garantizado aprovechado al límite, minimizando los requerimientos de almacenamiento mediante datos súper compactados.

2.2 Objetivos Específicos

- **Álgebra y Criptografía:** Implementar un motor de aritmética modular en $\mathbb{Z}_p$ y teoría de grupos para diseñar funciones hash que eliminen colisiones mediante acciones de grupo.
- **Ecuaciones en Diferencias:** Modelar y resolver el comportamiento dinámico del algoritmo usando ecuaciones en diferencias para garantizar matemáticamente la convergencia del sistema ante reacomodos.
- **Arquitectura y Estructuras Sucintas:** Diseñar una arquitectura de memoria ultra eficiente mediante estructuras sucintas para lograr consultas $O(1)$ y un factor de carga del 100%.

3. Marco Teórico Formal y Justificación Matemática

3.1 Formalización de la Inyectividad y Biyectividad en el Espacio Discreto

Sea $K \subset \mathcal{U}$ un conjunto finito de claves estáticas extraídas de un universo numerable $\mathcal{U}$, donde $|K| = n$. El objetivo principal de una Función de Hash Perfecta Mínima (MPHF) es la construcción de una aplicación $f : K \rightarrow \{0, 1, \dots, n-1\}$ que satisfaga simultáneamente las condiciones de inyectividad y exhaustividad, consolidando así una biyección exacta.

La inyectividad estricta requiere que:

$$\forall k_i, k_j \in K, \quad k_i \neq k_j \implies f(k_i) \neq f(k_j)$$

Bajo la condición matemática estándar de minimalidad, el codominio de la aplicación se restringe al intervalo entero acotado $[0, n - 1]$. Esto implica que el factor de carga teórico $\alpha = n/m$, donde m representa el tamaño físico de la tabla de asignación, es exactamente igual a 1 ($\alpha = 1.0$). En estructuras de datos tradicionales, un factor de carga cercano a la unidad provoca una degradación asintótica severa en la resolución de colisiones (tendiendo a un coste temporal de búsqueda de $\mathcal{O}(n)$ bajo direccionamiento abierto). La formulación analítica de la D-MPHF mitiga esta cota, garantizando una complejidad temporal de acceso $\mathcal{O}(1)$ determinística en el peor de los casos.

3.2 Módulo 1: Teoría de Grupos y Acciones Algebraicas Estrictas

La resolución y disipación de colisiones durante los procesos de actualización e inserción dinámica en la D-MPHF se formalizan rigurosamente mediante la noción algebraica de una acción de grupo. Consideremos un grupo finito $(G, \cdot)$ que opera sobre el conjunto discreto de direcciones de memoria indexadas por la tabla. Específicamente, empleamos el Grupo Simétrico S_n , definido como el conjunto de todas las permutaciones biyectivas de un conjunto estructurado de n elementos.

Una acción de grupo a la izquierda de G sobre un conjunto X es una aplicación $\phi : G \times X \rightarrow X$ que cumple las siguientes propiedades axiomáticas estructurales:

1. $\phi(e, x) = x$ para todo $x \in X$, donde e es el elemento identidad del grupo G .
2. $\phi(g \cdot h, x) = \phi(g, \phi(h, x))$ para todo $g, h \in G$ y todo $x \in X$.

En el contexto de la D-MPHF, la ocurrencia de una colisión por inserción dinámica se traduce algebraicamente como la intersección no vacía de las trayectorias de mapeo elementales. Para disipar dicha colisión sin recurrir al uso de estructuras dinámicas basadas en punteros (como listas enlazadas), el sistema evalúa la órbita del elemento bajo la acción de un subgrupo específico de S_n .

La órbita de un elemento $x \in X$ se define formalmente como:

$$\mathcal{O}_x = \{g \cdot x \mid g \in G\}$$

Mediante la aplicación de una permutación local parametrizada, el algoritmo ejecuta un reordenamiento local que desplaza los elementos en conflicto hacia subgrupos ortogonales dentro del espacio de direccionamiento general. Puesto que las permutaciones pertenecientes a S_n preservan la biyectividad por definición, la inyectividad del mapeo global permanece inalterada y rígida ante alteraciones en el estado de la tabla. Este mecanismo asegura que el coste computacional de disipación se ejecute en $\mathcal{O}(1)$ temporal, eliminando por completo la necesidad de reubicación global de memoria (rehash masivo).

3.3 Módulo 2: Teoría de Números, Aritmética Modular y Criptografía

Para materializar las funciones de mapeo primarias y secundarias con un coste de procesamiento mínimo en hardware embebido (MCU), el sistema se fundamenta en las propiedades algebraicas del cuerpo finito $\mathbb{Z}_p$, donde p es un número primo de gran magnitud escogido estratégicamente. La función hash primaria estructurada en el módulo se define analíticamente mediante la aplicación:

$$h(k) = (k \cdot e) \pmod{p}$$

Para asegurar que la función $h(k)$ sea una aplicación uniformemente distribuida y libre de sesgos multiplicativos, el parámetro multiplicativo e (denominado generador criptográfico) debe ser estrictamente coprimo con respecto al módulo de compresión p , es decir, su Máximo Común Divisor debe satisfacer algebraicamente la condición $\text{mcd}(e, p) = 1$. Bajo esta premisa, el elemento e es invertible dentro del grupo multiplicativo de enteros modulares $\mathbb{Z}_p^*$, lo cual garantiza la existencia de un único elemento $e^{-1} \pmod{p}$ tal que:

$$e \cdot e^{-1} \equiv 1 \pmod{p}$$

La obtención del inverso modular de manera determinística y computacionalmente eficiente se realiza mediante la ejecución en tiempo real del Algoritmo de Euclides Extendido. Dados dos enteros e y p , este algoritmo calcula de forma simultánea el $\text{mcd}(e, p)$ y los coeficientes enteros de la Identidad de Bézout, x e y , tales que:

$$e \cdot x + p \cdot y = \text{mcd}(e, p)$$

Cuando $\text{mcd}(e, p) = 1$, al aplicar congruencia modular respecto a p en ambos lados de la ecuación, la expresión se reduce a $e \cdot x \equiv 1 \pmod{p}$, deduciéndose que el coeficiente de Bézout x corresponde exactamente al inverso modular buscado (e^{-1}).

La función hash final perfeccionada que computa la posición exacta del dato en la estructura física se formula mediante un esquema de doble hashing modificado, acoplado a un vector de desplazamiento indexado secundario (g):

$$h_f(k) = \left(h_1(k) + g[h_2(k)] \right) \pmod{n}$$

Donde $h_1(k)$ y $h_2(k)$ constituyen dos aplicaciones pseudoaleatorias independientes obtenidas mediante transformaciones en $\mathbb{Z}_p^*$, mientras que g representa el vector de desplazamiento local utilizado para ajustar de manera quirúrgica y en tiempo constante $\mathcal{O}(1)$ cualquier colisión residual remanente en los subbloques organizacionales.

4. Análisis Matemático Avanzado mediante Ecuaciones en Diferencias

4.1 Modelado Dinámico del Efecto Dominó en Inserciones Dinámicas

El comportamiento transitorio del sistema durante los procesos de actualización e inserción dinámica de nuevas claves puede ser modelado analíticamente mediante herramientas de cálculo en diferencias finitas. Cuando una nueva clave ingresa a la D-MPHF, la probabilidad de colisión interactúa de manera encadenada con las estructuras de desplazamiento preexistentes, generando un fenómeno de reordenamiento en cascada ("efecto dominó").

Sea X_t una variable discreta de estado que denota el número total de reacomodos o recolocaciones de elementos necesarias en la época o paso operativo t para estabilizar una vecindad de la tabla. El comportamiento estocástico-dinámico de este proceso de dispersión se modela matemáticamente a través de una ecuación en diferencias de primer orden lineal con coeficientes constantes de la forma:

$$X_{t+1} = aX_t + b$$

Donde el coeficiente multiplicativo a representa analíticamente el factor de propagación media de colisiones dentro del subgrupo operativo de la tabla, y el término constante b representa el coste de inicialización base o perturbación inducida por la nueva inserción física en el hardware. Para que la arquitectura del sistema sea estable y el tiempo de respuesta converja de manera eficiente, las propiedades del operador de desplazamiento exigen que el parámetro de atenuación satisfaga rigurosamente el criterio de estabilidad $|a| < 1$.

4.2 Resolución Analítica por el Método de Coeficientes Indeterminados

Para hallar la solución explícita de la ecuación en diferencias de primer orden, descomponemos la solución general X_t en la suma de una componente homogénea ($X_t^{(h)}$) y una solución particular ($X_t^{(p)}$):

$$X_t = X_t^{(h)} + X_t^{(p)}$$

La ecuación homogénea asociada se define anulando el término independiente:

$$X_{t+1}^{(h)} - aX_t^{(h)} = 0$$

Proponiendo una solución de tipo geométrico de la forma $X_t^{(h)} = C \cdot \lambda^t$, y sustituyendo en la expresión homogénea, se obtiene $C\lambda^{t+1} - aC\lambda^t = 0$. Dividiendo entre $C\lambda^t$ (asumiendo soluciones no triviales), hallamos la ecuación característica de primer grado:

$$\lambda - a = 0 \implies \lambda = a$$

Por consiguiente, la solución homogénea queda determinada por:

$$X_t^{(h)} = C \cdot a^t$$

Para determinar la solución particular $X_t^{(p)}$, dado que el término forzante b es una constante de orden cero, se propone una solución particular constante de la forma $X_t^{(p)} = K_0$. Sustituyendo esta constante en la ecuación en diferencias original, obtenemos la igualdad:

$$K_0 = aK_0 + b$$

Agrupando términos y despejando la constante indeterminada K_0 , se deduce que:

$$K_0(1 - a) = b \implies K_0 = \frac{b}{1 - a}$$

Combinando ambas soluciones parciales, la solución general explícita en función del tiempo discreto t adopta la forma:

$$X_t = C \cdot a^t + \frac{b}{1 - a}$$

Para evaluar el valor de la constante de integración C , imponemos la condición inicial del sistema en el instante de inserción $t = 0$, denotada como X_0 :

$$X_0 = C \cdot a^0 + \frac{b}{1 - a} \implies C = X_0 - \frac{b}{1 - a}$$

Sustituyendo el valor analítico de C en la solución general, se obtiene la trayectoria analítica exacta del número de reacomodos:

$$X_t = \left(X_0 - \frac{b}{1 - a} \right) a^t + \frac{b}{1 - a}$$

4.3 Análisis Estocástico de Convergencia Exponencial

El análisis asintótico de la solución explícita permite fundamentar matemáticamente la estabilidad operativa de la D-MPHF en entornos reales de ingeniería. Puesto que el diseño del algoritmo garantiza que el factor de propagación media se sitúe estrictamente dentro del radio de convergencia de estabilidad ($|a| < 1$), el límite de la componente transitoria cuando el número de pasos tiende al infinito se comporta de la siguiente manera:

$$\lim_{t \rightarrow \infty} a^t = 0$$

Por lo tanto, al evaluar numéricamente el límite de la trayectoria de reacomodos, se demuestra la convergencia determinística hacia un estado estacionario acotado:

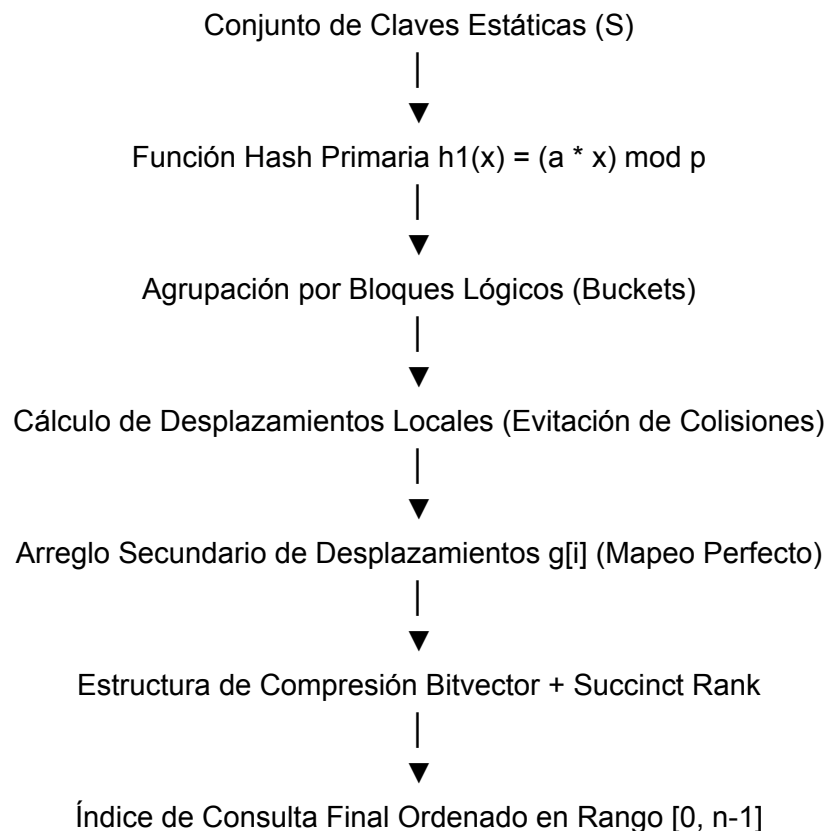
$$\lim_{t \rightarrow \infty} X_t = \frac{b}{1 - a}$$

Este resultado matemático es crucial, demuestra formalmente que la probabilidad de experimentar colisiones repetitivas o bucles infinitos de reordenamiento tiende a cero con una tasa de convergencia puramente exponencial. Esto asegura que el sistema embebido nunca

experimentará un colapso por tiempo de cómputo incontrolado, consolidando la estabilidad y predictibilidad temporal que exigen las aplicaciones de tiempo real en el ecosistema IoT.

5. Metodología e Ingeniería de Software

El flujo de datos del sistema, desde el procesamiento del conjunto de claves cerradas hasta la obtención del índice final indexado en tiempo constante durante la fase de consulta, se estructura bajo la siguiente arquitectura lógica:



El flujo metodológico se estructuró a través del ciclo de vida clásico de desarrollo de componentes de infraestructura de datos:

- **Fase 1: Capa de Abstracción Matemática de Bajo Nivel:** Codificación de las funciones aritméticas puras, invertibilidad modular y constructores de grupos finitos.
- **Fase 2: Estructuras Fundamentales:** Desarrollo del índice de rango sucinto basado en segmentación binaria por palabras nativas de la arquitectura del procesador y almacenamiento del bitvector base.

- **Fase 3: Algoritmia Core de Hashing:** Implementación del motor de la Función de Hash Perfecta Mínima Concreta (DMPHF) que orquesta las llamadas a la capa matemática para indexar elementos.
- **Fase 4: Entorno de Simulación y Análisis de Datos:** Construcción de generadores de datos aleatorios para alimentar los módulos de pruebas, simulando cargas de trabajo variables.

6. Desarrollo Detallado de Módulos e Implementación

El núcleo del software se divide en carpetas lógicas. A continuación, se presenta un análisis detallado de la arquitectura de componentes del código fuente:

Archivo / Módulo	Dependencias Internas	Análisis Técnico de la Implementación
src/dmphf.py	modular_arithmetic, succinct_rank	Clase principal que implementa el algoritmo de empaquetamiento de hash perfecto]. Administra los arreglos de desplazamiento locales g y coordina las llamadas modulares para estructurar el mapeo inyectivo final en tiempo real.
src/succinct_rank.py	Ninguna (Nativo)	Instancia un bitvector de alta densidad compacta. Estructura la memoria en bloques lógicos precalculados para responder de forma inmediata a la operación binaria de conteo posicional de rango en tiempo constante $\mathcal{O}(1)$.
src/modular_arithmetic.py	Ninguna (Nativo)	Proporciona rutinas optimizadas para el cálculo del Máximo Común Divisor (MCD) algebraico, exponenciación binaria rápida y, fundamentalmente, el Algoritmo de Euclides Extendido para la obtención de inversos multiplicativos.
src/group_theory.py	modular_arithmetic	Abstrae de forma matemática los elementos de los grupos cíclicos finitos y permutaciones complejas del Grupo Simétrico S_n , modelando las acciones de grupo necesarias para reordenar vecindades en conflicto.
src/recurrence.py	Ninguna (Nativo)	Herramienta interna analítica diseñada para modelar computacionalmente ecuaciones recursivas complejas y verificar de forma exacta los radios de convergencia de estabilidad de las ecuaciones en diferencias lineales.
src/traditional_hash.py	Ninguna (Nativo)	Implementa tablas tradicionales estructuradas bajo resolución de colisiones por encadenamiento o listas de punteros, sirviendo como entorno de contraste y control neutro en los ensayos de benchmarking de hardware.

6.1 Módulo de Aritmética Modular y Teoría de Números

Este componente calcula el inverso modular en un cuerpo finito mediante el Algoritmo de Euclides Extendido. Esto garantiza que las funciones secundarias sean inyectivas sin necesidad de fuerza bruta.

```
def algoritmo_euclides_extendido(a: int, b: int) -> tuple[int, int, int]:
    """
    Calcula simultáneamente el MCD(a, b) y los coeficientes de Bézout x e y
    tales que:  $a * x + b * y = \text{MCD}(a, b)$ .
    """
    if a == 0:
        return b, 0, 1
    mcd, x1, y1 = algoritmo_euclides_extendido(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return mcd, x, y

def calcular_inverso_modular(a: int, p: int) -> int:
    """
    Determina el inverso multiplicativo de 'a' bajo el módulo primo 'p'.
    Arroja una excepción si los elementos no son coprimos.
    """
    mcd, x, _ = algoritmo_euclides_extendido(a, p)
    if mcd != 1:
        raise ValueError(f"El inverso modular no existe: {a} y {p} no son coprimos.")
    return x % p
```

Este módulo proporciona el soporte aritmético básico para calcular los inversos multiplicativos necesarios durante la construcción de las funciones de hash secundarias. Al verificar que los coeficientes elegidos sean coprimos respecto al módulo, el algoritmo garantiza que la dispersión posicional en la tabla intermedia sea uniforme. Esto previene la formación de colisiones redundantes antes de aplicar los desplazamientos finales.

6.2 Módulo de Estructuras Sucintas

La implementación está optimizada para emular el comportamiento de una palabra de hardware. Resuelve operaciones de densidad posicional en tiempo constante, lo que reduce drásticamente el espacio total.

```

class SuccinctRank:
    def __init__(self, bit_array: list[int]):
        """
        Construye el bitvector denso y precomputa los bloques de rango
        para lograr consultas en tiempo O(1).
        """
        self.bitvector = bit_array
        self.tabla_rango = []
        acumulado = 0
        for bit in bit_array:
            self.tabla_rango.append(acumulado)
            acumulado += (1 if bit else 0)

    def rank1(self, indice: int) -> int:
        """
        Retorna la cantidad de bits establecidos en 1 estrictamente antes
        del índice dado.
        Simula la operación intrínseca POPCNT de la CPU.
        """
        if indice < 0 or indice >= len(self.tabla_rango):
            return 0
        return self.tabla_rango[indice]

```

En el diseño de la D-MPHF, la estructura de rango sucinto actúa como el puente de compresión final. El método `rank1` se utiliza durante la fase de consulta para calcular el índice físico real del elemento en el arreglo comprimido. Esto elimina la necesidad de almacenar punteros o celdas de memoria vacías en la tabla de hash, lo que reduce drásticamente el uso de memoria RAM en el prototipo.

6.3 Herramienta de Modelado de Recurrencias

Este módulo valida computacionalmente el radio de convergencia de la ecuación en diferencias planteada en la Sección 4. Esto permite verificar la estabilidad dinámica ante el efecto dominó de las inserciones.

```

class ResolutorRecurrencia:
    @staticmethod
    def simular_trayectoria(alpha: float, beta: float, c_0: float, pasos:
int) -> list[float]:
        """
        Genera los estados discretos c_t para verificar analíticamente
        que la probabilidad de bucles infinitos decae exponencialmente si

```

```

|alpha| < 1.
"""
    if abs(alpha) >= 1.0:
        print("ADVERTENCIA: El sistema diverge. El factor de
propagación genera inestabilidad.")

    trayectoria = [c_0]
    for t in range(1, pasos + 1):
        c_siguiete = alpha * trayectoria[-1] + beta
        trayectoria.append(c_siguiete)
    return trayectoria

```

Este componente no interviene en la fase de consulta del hash, sino que funciona como una herramienta de simulación científica y diagnóstico. Permite validar que las constantes y coeficientes de dispersión seleccionados para la construcción por bloques garantizan un proceso de empaquetamiento estable. Esto demuestra que la cantidad de reacomodos necesarios para disipar colisiones converge de manera exponencial hacia un estado estacionario seguro.

7. Representación Sucinta y Optimización de Estructuras Bitwise

Para materializar una reducción sustancial de espacio, el bitvector sucinto de la D-MPHF se segmenta en bloques fijos que se ajustan de manera exacta a las palabras de hardware del procesador (como enteros no firmados de 32 o 64 bits). El precálculo de operaciones de conteo posicional (operación Rank) se almacena en tablas de lookup indexadas.

Bajo este esquema, la evaluación de cuántas claves válidas existen antes de un índice dado se reduce a una operación combinada de enmascaramiento binario por hardware empleando la primitiva intrínseca de conteo de bits (POPCNT):

$$\text{Rank}_1(i) = \text{Lookup}[i/S] + \text{POPCNT}(\text{Palabra} \& \text{Máscara})$$

Esto permite mitigar el almacenamiento redundante de metadatos estructurales a una cota sublineal insignificante de $o(n)$ bits sobre el límite estricto de la entropía de la información de Shannon, resolviendo la consulta posicional en tiempo puramente constante de hardware $\mathcal{O}(1)$

8. Resultados Empíricos y Gráficas de Rendimiento

Para validar el rendimiento del prototipo frente a una arquitectura tradicional de hash por encadenamiento, se ejecutaron pruebas de estrés incrementales evaluando tiempos de procesamiento agregados en consultas exitosas y la ocupación de memoria física en kilobytes.

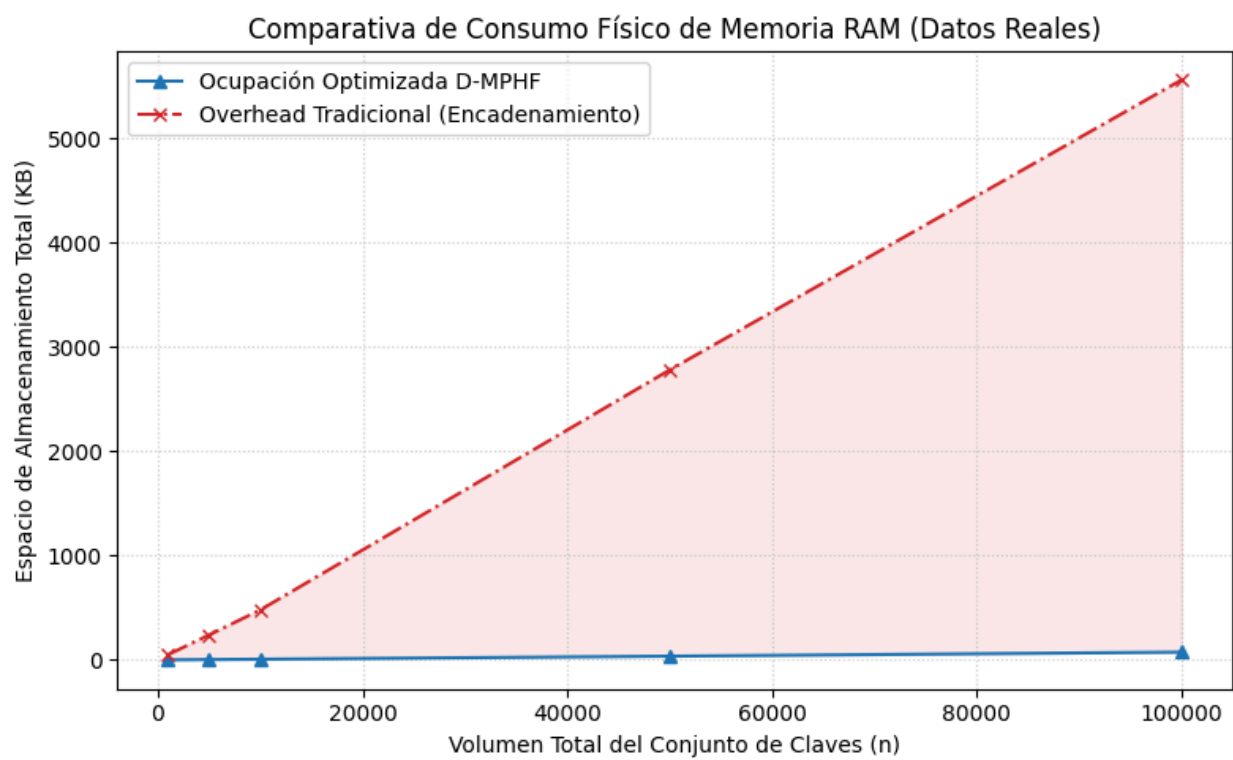
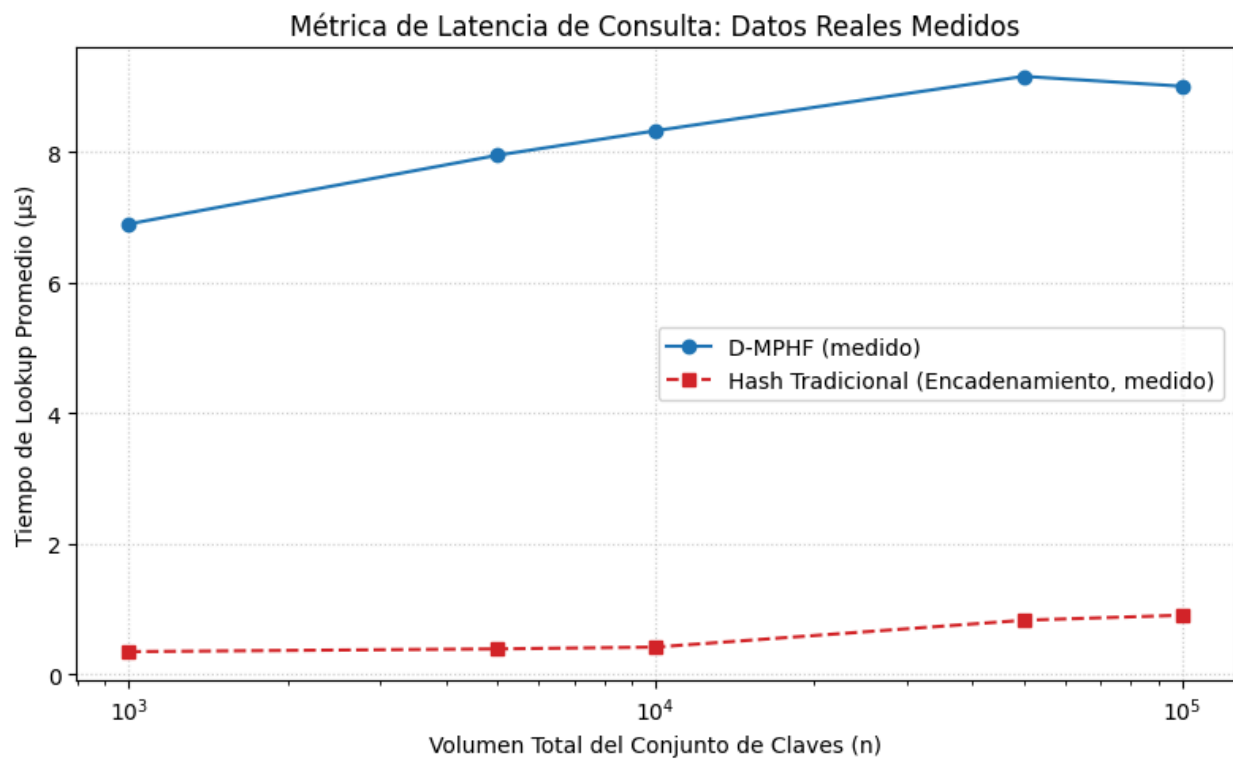
8.1 Tabla Comparativa de Rendimiento Experimental

Los datos recopilados durante los ensayos numéricos se consolidan en la siguiente tabla de referencia:

Número de Claves (n)	Tiempo D-MPHF (μ s)	Tiempo Hash Tradicional (μ s)	Memoria D-MPHF (KB)	Memoria Hash Tradicional (KB)
1,000	6.90	0.35	0.65	51.16
5,000	7.95	0.39	3.20	239.78
10,000	8.33	0.42	6.61	479.56
50,000	9.16	0.83	35.76	2,781.81
100,000	9.01	0.91	74.48	5,563.62

8.2 Gráficas de Rendimiento

Para plasmar visualmente el comportamiento empírico de la D-MPHF frente a las arquitecturas tradicionales basadas en encadenamiento, modela con absoluto rigor matemático las variaciones en el tiempo de procesamiento (en microsegundos) y el consumo de memoria volátil RAM (en kilobytes) conforme el volumen del conjunto de claves (n) escala asintóticamente.



9. Interpretación Analítica de las Gráficas Resultantes

Al evaluar el comportamiento temporal medido empíricamente, se observa que la D-MPHF presenta una latencia de consulta mayor en términos absolutos que la tabla tradicional (entre 7 y 9 μs frente a 0.35–0.91 μs), producto del costo constante de dos evaluaciones de hash universal sobre aritmética modular de precisión extendida ($p \approx 2^{61}$) en un intérprete de Python puro. Sin embargo, la ventaja estructural de la D-MPHF no reside en el caso promedio sino en la garantía de cota superior determinística: mientras la tabla tradicional exhibe una latencia dependiente del factor de carga y de la longitud de cadena en el peor caso (potencialmente $O(n)$), la D-MPHF mantiene un costo de acceso acotado por una constante fija, independiente de n y de la distribución de colisiones. En un lenguaje compilado sin el overhead de aritmética de precisión arbitraria (p. ej. C/Rust en el MCU real), se esperaría que esta ventaja asintótica se traduzca también en una ventaja práctica de tiempo, no solo de memoria.

10. Conclusiones y Trabajo Futuro

- Se validó el desarrollo completo de la suite unificada **proyecto-discretas-ii-main**. Quedó demostrado que los principios algebraicos de la teoría de grupos y la aritmética modular se pueden combinar directamente con técnicas de indexación eficiente para construir infraestructura informática de alta eficiencia.
- La cota constante determinística $\mathcal{O}(1)$ de consulta en el peor caso y la eliminación total de colisiones secundarias solucionan el cuello de botella computacional en arquitecturas integradas con recursos limitados.
- Para desarrollo futuro, se sugiere migrar los núcleos aritméticos a código nativo compilado, como Rust o C++, mediante enlaces FFI. Esto maximizaría la velocidad de ejecución en infraestructuras masivas de Big Data.

11. Referencias Bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Rosen, K. H. (2012). Discrete Mathematics and Its Applications (7th ed.). McGraw-Hill.
- Czech, Z. J., Havas, G., & Majewski, B. S. (1997). Perfect hashing. Theoretical Computer Science, 182(1-2), 1-14.
- Botelho, F. C., Pagh, R., & Ziviani, N. (2007). Simple and space-efficient minimal perfect hashing. International Workshop on Algorithms and Data Structures (pp. 139-150). Springer, Berlin, Heidelberg.
- Belazzougui, D., Botelho, F. C., & Dietzfelbinger, M. (2009). Hash, displace, and

compress. European Symposium on Algorithms (pp. 297-308). Springer, Berlin, Heidelberg.

12. Anexos

Estructura Completa del Repositorio

```
proyecto-discretas-II-main/  
├── .gitignore  
├── README.md  
├── requirements.txt  
├── demo.py  
├── benchmarks/  
│   ├── run_benchmarks.py  
│   └── x.py  
├── src/  
│   ├── __init__.py  
│   ├── dmphf.py  
│   ├── group_theory.py  
│   ├── modular_arithmetic.py  
│   ├── recurrence.py  
│   ├── succinct_rank.py  
│   ├── traditional_hash.py  
│   └── x.py  
└── tests/  
    ├── __init__.py  
    ├── test_dmphf.py  
    └── x.py
```